



Lista 05 - Algoritmos Gulosos

1. Projete um algoritmo para encontrar os k menores elementos de um conjunto não ordenados de n inteiros em tempo $O(n + k \lg n)$.
 2. Projete um algoritmo eficiente para calcular a união dos conjuntos não ordenados A e B , onde $n = \max(|A|, |B|)$. A saída deve ser um vetor de elementos distintos que formam a união dos conjuntos. O algoritmo projetado deve ter complexidade de tempo $O(n \lg n)$.
 3. Considere o conjunto numérico X com $2n$ elementos. O problema proposto consiste em particionar o conjunto X em dois conjuntos A e B , onde $|A| = |B| = n$, de forma que maximize o valor $\sum_{a \in A} a - \sum_{b \in B} b$. Desenvolva um algoritmo guloso para esse problema. O tempo do seu algoritmo deve ser de $O(n \lg n)$.
 4. Considere um conjunto de n livros, onde cada livro i , $1 \leq i \leq n$, possui peso p_i e $0 < p_i \leq 5$. O problema proposto consiste em dividir os n livros no menor número possível de envelopes de modo que cada envelope tenha no máximo 2 livros e o peso do conteúdo de cada envelope seja no máximo 5. Desenvolva um algoritmo guloso para esse problema. O tempo do seu algoritmo deve ser de $O(n \lg n)$.
 5. Considere o problema de escalonar n processos com tempos de execução $t_1, t_2, \dots, t_n$ conhecidos para serem executados por um processador com um único núcleo. Os processos podem ser executados em qualquer ordem, mas apenas um de cada vez. Você deseja escolher uma ordem que minimize o **tempo total de espera** de todos os processos. O **tempo de espera** de um processo é definido como a soma dos tempos dos processos que executam antes dele, e o tempo total de espera é a soma dos tempos de espera de cada processo. Desenvolva um algoritmo guloso para esse problema e demonstre que a estratégia gulosa usada sempre gera a solução ótima do problema.
- Definição 1.** Uma sequência Y é dita subsequência da sequência X , denotada por $Y \subseteq X$, se Y pode ser obtido a partir de X pela remoção de zero ou mais elementos, porém sem alterar a ordem relativa dos elementos restantes.
6. O problema da **Maior Subsequência Comum** pede para, dadas duas subsequências X e Y , determinarmos uma sequência Z que seja subsequência de ambas X e Y e que possua o máximo de elementos possível. Desenvolva um algoritmo guloso para esse problema e demonstre que a estratégia gulosa usada sempre gera a solução ótima do problema.
 7. Considere o trecho da BR-122 entre Quixadá e Quixeramobim, onde as casas ao longo das margens se distribuem bem esparsamente. Os moradores dessa região, para não ficarem fora das novidades do mundo, são ávidos usuários de celulares. Uma empresa de telecomunicações deseja espalhar suas antenas em certos pontos ao longo dessa rodovia para atender os clientes dessa região de forma que todos estejam a um máximo de $4km$ de alguma antena.



Forneça um algoritmo que, dada a quantidade n de residências e as distâncias de cada uma delas, em km , até o ponto inicial do trecho, determine em que os pontos devem ser instaladas as antenas de forma minimizar a quantidade necessária.

8. Existem n caiaquistas que precisamos distribuir em caiaques de dois lugares de forma a respeitar o peso máximo suportado pelo caiaques. Desejamos usar o mínimo de caiaques possível. Você deve considerar que:

- O peso máximo suportado por cada caiaque é K ;
- O caiaquista i , com $1 \leq i \leq n$, possui um peso p_i ;
- Nenhum caiaquista pesa mais que o máximo suportado por um caiaque.

Desenvolva um algoritmo guloso para esse problema e demonstre que a estratégia gulosa usada sempre gera a solução ótima do problema.

9. Projete um algoritmo guloso para o problema da Mochila Fracionária. O problema da Mochila Fracionária pode ser definida como dada uma mochila de capacidade M e n objetos, onde cada objeto possui um peso $p_i \leq M$ e um valor v_i , desejamos descobrir qual a fração $0 \leq x_i \leq 1$ de cada objeto i , $1 \leq i \leq n$, iremos selecionar de forma que maximize o valor $\sum_{i=1}^n x_i \cdot v_i$. Lembre-se que a

mochila possui uma limitação de peso M , ou seja, $\sum_{i=1}^n x_i \cdot p_i \leq M$.

10. Você e seus amigos vão começar uma empresa de segurança e precisam obter licenças das autoridades de segurança para compor as partes do software que criaram. Por conta da regulamentação, as empresas que não quiserem compartilhar com o governo detalhes do funcionamento do que estão desenvolvendo ficam limitadas a adquirir apenas uma licença por mês. Preocupados com questões privacidade e de direitos, vocês decidem que a melhor opção é realmente se submeter às limitações.

Cada licença atualmente está atrelada a uma custo de R\$ 100,00. Entretanto, os custos com cada licença aumentam a cada mês de forma exponencial: em particular, o custo de uma licença j aumenta por um fator $r_j > 1$ a cada mês. Isso significa que se a licença j for comprada daqui a t meses, ela custará R\$ $100,00 \cdot r_j^t$. Vamos assumir que os fatores de aumento de cada licença são distintos entre si.

Obedecendo as restrições, dado o custo inicial R\$ 100,00, a quantidade de n licenças e os fatores de aumento de licenças que vocês precisam adquirir, em que ordem devemos comprar as licenças de forma a minimizar o custo total de sua aquisição? Forneça um algoritmo guloso que determina essa ordem.

11. A superpopular ferramenta de buscas Q? precisa realizar uma grande quantidade de processamento a cada vez que precisa recompilar seu índice. Felizmente, a companhia dispõe de um grande supercomputador, junto a uma quantidade essencialmente ilimitada de PCs de ótima configuração. Eles dividiram a computação em geral em n processamentos distintos, rotulados $J_1, J_2, \dots, J_n$, que podem ser realizados de forma completamente independente entre si. Cada tarefa de processamento consiste de dois estágios: um préprocessamento no supercomputador e uma finalização em algum



PC, cada um consumindo uma certa quantidade de tempo dependendo da tarefa. Vamos dizer que J_i necessita de p_i segundos no supercomputador seguidos de f_i segundos em um PC.

Já que existem mais que n PCs disponíveis na empresa especificamente para esta atividade, as finalizações das tarefas podem ser realizadas completamente em paralelo. Entretanto, o supercomputador só pode trabalhar em uma tarefa por vez, sem preempção, de forma que os engenheiros precisam determinar uma ordem na qual entregar as tarefas a ele. Assim que uma tarefa, na ordem, for finalizada, ela poderá ser delegada a algum PC; nesse momento, a próxima tarefa pode ser enviada ao supercomputador.

Forneça um algoritmo guloso para determinar uma ordem na qual o tempo final do processamento seja minimizado. O tempo final é o tempo desde o início da recompilação até o momento em que todas as tarefa tenham finalizado suas execuções.

12. Você está fazendo consultoria para uma empresa de transporte de cargas que lida com uma grande quantidade de entregas entre Fortaleza e Quixadá. O volume é grande o suficiente para obrigá-los a enviar mais de um caminhão por dia entre as duas cidades.

Os caminhões possuem um peso máximo W que podem transportar. As caixas que transportam chegam no depósito de Fortaleza uma a uma, e cada caixa i possui peso w_i . A estação de carregamento tem um espaço limitado, de forma que apenas um caminhão cabe na estação por vez.

Uma das políticas da empresa é de que os pacotes devem ser carregados nos caminhões na ordem em que chegarem. No momento, a empresa está usando uma estratégia simples para o carregamento das caixas: eles carregam o caminhão com as caixas que couberem. Assim que a próxima caixa não couber, eles despacham o caminhão, colocam a caixa no próximo caminhão e continuam o carregamento. Eles estão preocupados em estarem usando caminhões demais e gostariam de sua opinião sobre se a estratégia pode ser melhorada respeitando as políticas da empresa. Mostre que, dada a restrição, a forma atual de empacotamento sempre necessita da menor quantidade de caminhões possível para transportar n caixas com pesos w_j .